

Essentials of Data Science Laboratory - 2304102L - 2026 - 2304102L

Dashboard Contents Calendar Assessments Syllabus

- Search course ctrl+k
- 1. Practical 1
 - 2. Practical 2
 - 3. Practical 3
 - 4. Practical 4
 - 5. Practical 5



Pickup from where you left off!

Titanic Dataset

Practical 5 • Lab Assignment

Resume

Practical 1

Unit • 100% completed

Practical 2

Unit • 100% completed

Practical 3

Unit • 100% completed

Practical 4

Unit • 100% completed

Practical 5

Unit • 100% completed

03:01 🔊 🌙 ✎ 🔗 —

v is the velocity of the object (in meters per second).

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

+

Explorer

2 out of 2 hidden test case(s) passed

50.00kgm/s

Test cases

1.1.2. Conditional Calculation Based on the Number of Digits

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:
 $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer n .

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Sample Test Cases

condition...

```
1 #write your code here...
2 import math
3 number = int(input())
4 if 1 <= number <= 9:
5     print(number**2)
6 elif 10 <= number <= 99:
7     print(f"{math.sqrt(number):.2f}")
8 elif 100 <= number <= 999:
9     print(f"{math.cbrt(number):.2f}")
10 else:
11     print("Invalid")
```

Average time: 0.006 s (6.29 ms) | Maximum time: 0.008 s (8.00 ms)

4 out of 4 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case	Expected output	Actual output
Test case 1	9 81	9 81
Test case 2		
Test case 3		

Terminal | Test cases

1.1.3. Days between Two Dates

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format *YYYY – MM – DD*.
- The second line contains the second date in the format *YYYY – MM – DD*.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

noOfDay...

```
1 from datetime import date
2 from datetime import datetime
3 date1 = input()
4 date2= input()
5 D1=datetime.strptime(date1,"%Y-%m-%d")
6 D2=datetime.strptime(date2,"%Y-%m-%d")
7 days=(D2-D1).days
8 print(days)
```

Average time

0.013 s

12.75 ms

Maximum time

0.033 s

33.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 33 ms

Debug

Expected output

2014-07-02

2014-11-02

123

Actual output

2014-07-02

2014-11-02

123

Test case 2 6 ms

Terminal

Test cases

1.1.4. Reverse a Number

01:14

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases

+

reverseN...

Submit

```
1 #write your code here...
2 num = int(input())
3 reversed_num = int(str(num)[::-1])
4 print(reversed_num)
```

Terminal

Test cases

1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases

multipl...

Submit

1 #write your code here...
2 num = int(input())
3 for i in range(1,11):
4 print(f"{num} x {i} = {num*i}")
5
6
7

Average time
0.009 s
9.00 ms

Maximum time
0.011 s
11.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 11 ms Debug

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40

Terminal

Test cases

1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

passorFa...

```
1 n = int(input())
2
3 marks = list(map(int, input().split()))
4
5 if len(marks) != n:
6     print("Invalid input")
7 elif any(m < 40 for m in marks):
8     print("Fail")
9 else:
10    percentage = sum(marks) / n
11    print(f"Aggregate Percentage: {percentage:.2f}")
12
13    if percentage > 75:
14        grade = "Distinction"
15    elif percentage >= 60:
16        grade = "First Division"
17    elif percentage >= 50:
18        grade = "Second Division"
19    else:
20        grade = "Third Division"
21
22    print(f"Grade: {grade}")
```

Terminal Test cases

1.2.2. Fibonacci Series

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

```
1 def fibonacci(n):
2
3     if n <= 1:
4         return 0
5     if n==2:
6         return 1
7     else:
8
9         return fibonacci(n-1) + fibonacci(n-2)
10
11
12 n = int(input())
13 for i in range(1, n + 1):
14     print(fibonacci(i), end=" ")
15
```

Terminal Test cases

05:03

Input Format:

- The input is an integer, representing the number of rows in the pattern.

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

+

 rightangle...

 Submit

```
1 n = int(input())
2
3 for i in range(1, n + 1):
4     for j in range(i):
5         print('*', end=' ')
6     print()
```

Average time

0.006 s

5.83 ms

Maximum time

0.008 s

8.00 ms

2 out of 2 shown test case(s) passed

4 out of 4 hidden test case(s) passed

✔ Test case 1 8 ms

 Debug

Expected output

5

•

• • •

● ● ●

● ● ● ● ●

● ● ● ● ●

Actual output

5

• *Journal of Management Education*

• • •

● ● ●

一、二、三、四、

五、中、西、南、北、東

 Terminal

Test cases

1.2.4. Pattern - 2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases +

numberP...

1

n = int(input())

2

for i in range(1, n + 1):

3

for j in range(1, i + 1):

4

print(j, end=" ")

5

print()

6

7

8

Average time

0.005 s

4.75 ms

Maximum time

0.006 s

6.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

6 ms

Debug

Expected output

Actual output

5

5

1

1

1 2

1 2

1 2 3

1 2 3

1 2 3 4

1 2 3 4

1 2 3 4 5

1 2 3 4 5

Terminal

Test cases